UNIVERSIDAD ESTATAL PENÍNSULA DE SANTA ELENA



FACULTAD DE SISTEMAS Y TELECOMUNICACIONES
CARRERA DE SOFTWARE

MATERIA:
PROGRAMACIÓN FUNCIONAL

FORMATIVA:
SIMULADOR FUNCIONAL CON INTERFAZ GRÁFICA

DOCENTE:
ING. ANTHONY ABRAHAN PACHAY ESPINOZA, MSC.

ESTUDIANTE:
TOMALÁ TUMBACO MIKE NEIMAN

LA LIBERTAD – ECUADOR

2026

Repositorio del Proyecto:
https://github.com/mikeneiman19/gestor-tareas-funcional

1. Análisis funcional: Gestor de Tareas Funcional

1. Problema que resuelve

Es un gestor de tareas con interfaz gráfica. Permite crear tareas con una prioridad (alta, media o baja), completarlas, eliminarlas, completarlas todas a la vez y filtrarlas por prioridad. Muestra cuántas tareas pendientes hay en la vista actual y guarda un historial de eventos.

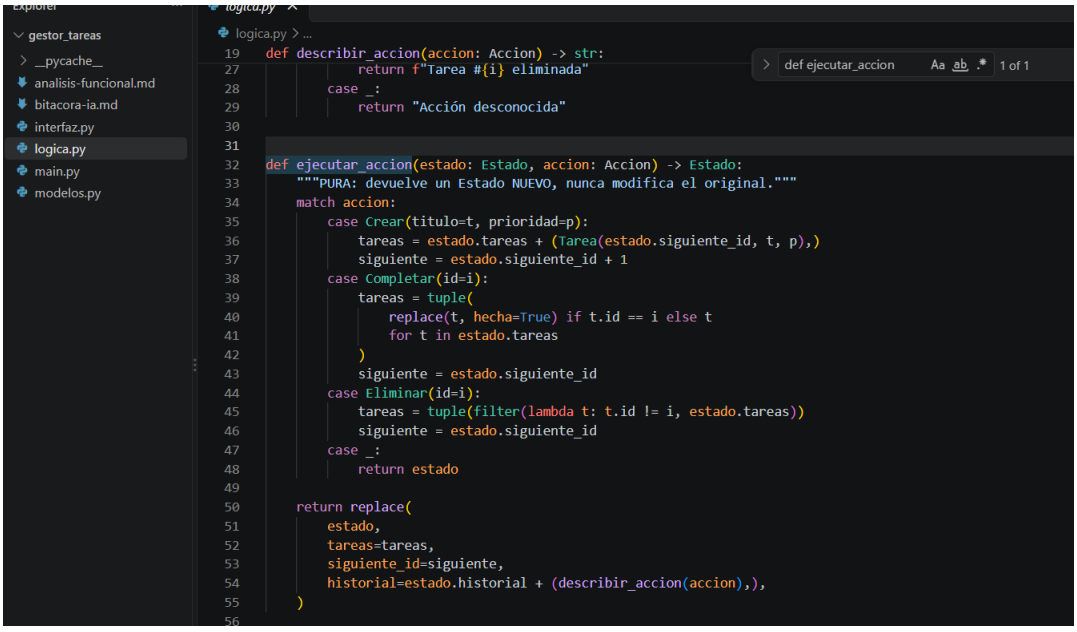
2. Estructuras de datos

- Tarea(dataclass frozen): id, titulo, prioridad, hecha.
- Crear, Completary Eliminar(dataclass frozen): las acciones del sistema.
- Accion = Union[Crear, Completar, Eliminar].
- Estado(dataclass frozen): tupla de tareas, tupla de historial y el siguiente id disponible.

Se usan tuplas en lugar de listas para que el estado no se pueda modificar con .append()u otras operaciones que mutan.

3. Dónde se aplicó la Unidad 1

- **Funciones de orden superior:** crear_filtro_prioridades de orden superior porque devuelve una función. map, filtery reducelo son porque reciben funciones como argumento. filtrar_por_prioridadllama a crear_filtro_prioridady le pasa el resultado a filter.
- **match/case:** en ejecutar_accionyen describir_accion. Cada tipo de acción (Crear, Completar, Eliminar) hace algo distinto, por eso match/casese justifica y no es un adorno.

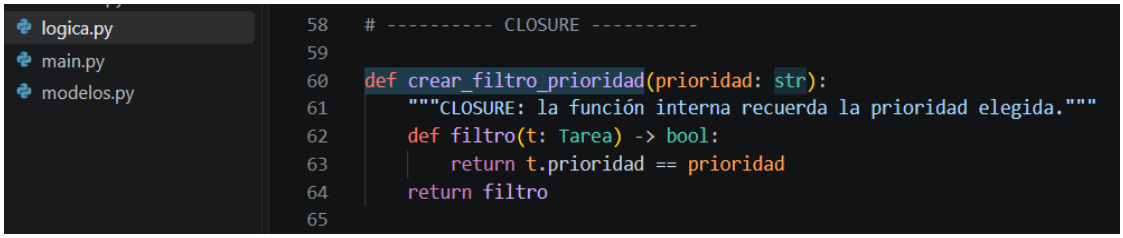


- **Modelado de datos:** dataclass con frozen=Truey Unionen modelos.py.

- **Funcional vs. imperativo:** contar_pendientes(con reduce) frente a contar_pendientes_imperativo(con fory una variable totalque cambia en cada vuelta).

4. Dónde se aplicó la Unidad 2

- **Lambda:** en map(completar_todas), en reduce(contar_pendientes), en filter(caso Eliminar) y en el binddel combo del filtro.
- **Closure:** crear_filtro_prioridad.

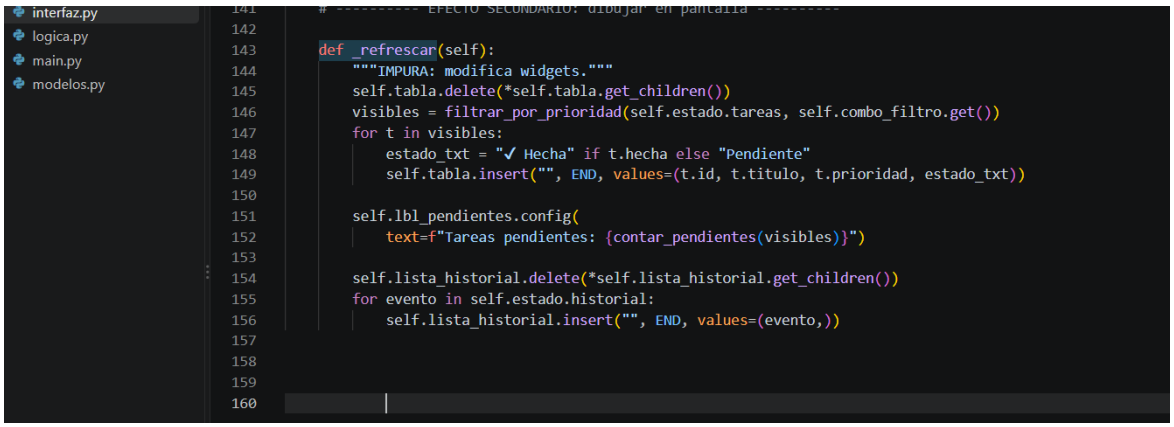


- **Inmutabilidad:** dataclass frozen, tuplas y replace().
- **Pureza:** toda la lógica de logica.py.
- **map/filter/reduce:** completar_todas(map), filtrar_por_prioridad(filter) y contar_pendientes(reduce).

5. Partes puras e impuras

Puras: estado_inicial, validar_titulo, describir_accion, ejecutar_accion, crear_filtro_prioridad, filtrar_por_prioridad, completar_todasy contar_pendientes. Son puras porque no hacen print, no leen archivos, no tocan la pantalla y no modifican nada externo: con la misma entrada dan siempre la misma salida.

Impuras: _refrescar(modifica widgets), _id_elegido(lee la selección de la tabla y puede mostrar un messagebox) y _titulo_valido(lee el campo de texto y puede mostrar un aviso).



6. Efectos secundarios

Dibujar la tabla y el historial, actualizar la etiqueta de pendientes, mostrar avisos con messagebox, y leer lo que el usuario escribe, selecciona o elige en los combos. Todos están en interfaz.py.

7. Estado inmutable

Todo el Estado. Cuando el usuario crea, completa o elimina una tarea, el estado no se modifica: ejecutar_accion devuelve un Estado nuevo con replace() y la interfaz reemplaza self.estado por ese nuevo valor. El estado anterior queda intacto, como se comprobó en la prueba de consola (e0 sigue con 0 tareas después de crear tareas en e1 y e2).

8. Closure

crear_filtro_prioridad(prioridad) devuelve una función filtro que recuerda la prioridad elegida aunque crear_filtro_prioridad ya terminó. Se usa en filtrar_por_prioridad para quedarse solo con las tareas de la prioridad que el usuario elige en el combo "Ver prioridad".

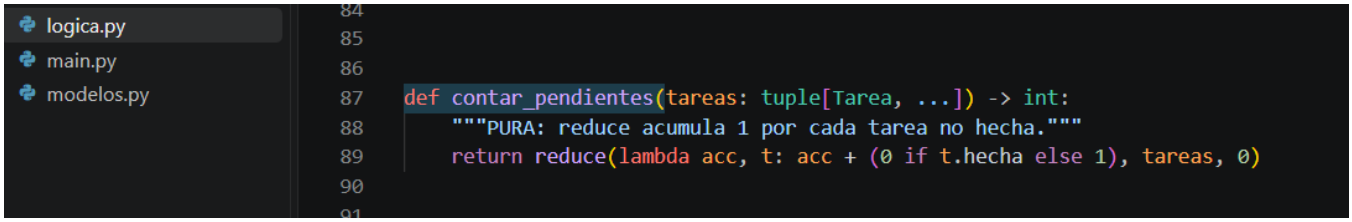
9. Qué sugirió la IA

La IA sugirió la estructura en tres capas (modelos, lógica e interfaz), las dataclasses, el uso de match/case, el closure crear_filtro_prioridad, el uso de map/filter/reduce, el código completo de la interfaz con ttkbootstrap y la idea de pasar de la Arena a un gestor de tareas cuando pregunté qué otro dominio podía haber elegido.

10. Qué corregí o rechacé

Decidí cambiar el contador de pendientes para que respete el filtro: antes contaba todas las tareas del estado, aunque estuvieran ocultas. Para eso cambié contar_pendientes y contar_pendientes_imperativo para que reciban una tupla de tareas en lugar del Estado, y en
_refrescar pasé visibles en vez de self.estado.

Al hacer el cambio apareció el error AttributeError: 'tuple' object has no attribute 'tareas'. La causa fue que cambié el parámetro de la función pero dentro del reduce quedó estado.tareas. Lo corregí cambiándolo por tareas, y volví a ejecutar logica.py y main.py para comprobar que funcionaba. Después de eso apareció un segundo error, TypeError: reduce() arg 2 must support iteration, porque en interfaz.py todavía llamaba a contar_pendientes(self.estado). Lo cambié por contar_pendientes(visibles).



11. Qué aprendí técnicamente

Aprendí que inmutabilidad significa no modificar un objeto sino crear uno nuevo con el cambio, y que por eso el estado anterior no se pierde. Aprendí que una función pura da siempre el mismo resultado y no hace print ni toca la pantalla, y que por eso conviene que el contador reciba solo las tareas que necesita y no todo el estado. Aprendí que un closure es una función que recuerda valores de la función que la creó, como la prioridad en crear_filtro_prioridad. También aprendí a leer un error de Python: el mensaje y el número de línea me indicaron exactamente que nombre faltaba cambiar. Y que separar la interfaz de la lógica permite probar la lógica sola en consola antes de abrir la ventana.

Por qué no conviene lambda en algunos casos

Una lambda con varias condiciones es ilegible, no se puede documentar y sus errores son poco claros.

MAL:

```
prioridad_valida = lambda t, p: t.prioridad == p if p != "Todas" and not t.hecha else True
```

BIEN:

```
def pasa_filtro(t, p):
    if p == "Todas":
        return True
    return t.prioridad == p
```

Para lógica con varias decisiones uso def. Las lambdas del proyecto son cortas, se usan una sola vez y están dentro de map, filter, reduceo de un bind.

2. Bitácora de uso de IA: Gestor de Tareas Funcional

Interacción 1: elegir otro dominio

- Prompt: "si hubiera elegido otra cosa para crear cuál hubiera sido?" y luego "si sería gestor de tarea cómo quedaría, me ayudas a crearlo?".
- Fragmento sugerido: la IA propuso migrar la estructura de la Arena a un gestor de tareas, con una tabla de equivalencias entre conceptos (dataclass, Union y match/case, closure, map, filter y reduce).
- Aceptado: la idea de usar la misma estructura de tres capas.
- Modificado: nada en esta etapa.
- Rechazado: nada.
- Por qué: la estructura ya la había probado con la Arena y me permitía reutilizar lo que entendí de las dos unidades.

Interacción 2: modelos.py y logica.py

- Prompt: el mismo pedido de crear el gestor de tareas.
- Fragmento sugerido: las dataclasses Tarea, Crear, Completar, Eliminar, la Union Accion y Estado; en la lógica, ejecutar_accion y describir_accion con match/case, el closure crear_filtro_prioridad, y filtrar_por_prioridad, completar_todas y contar_pendientes con filter, map y reduce, más la versión imperativa con for.
- Aceptado: todo el código. Ejecuté python logica.py y la salida fue la esperada (tareas en e0: 0, en e2: 2, pendientes 2 y 1, solo alta: ['Estudiar']).
- Modificado: más adelante cambié contar_pendientes (ver interacción 4).
- Rechazado: nada.
- Por qué: son funciones puras que devuelven un Estado nuevo en lugar de modificar el anterior, y la lógica queda separada de la interfaz.

Interacción 3: interfaz.py y main.py

- Prompt: seguí pidiendo ayuda con el gestor hasta tener los 4 archivos y luego ejecuté python main.py.
- Fragmento sugerido: la clase TareasApp con formulario de título y prioridad, combo de filtro, tabla de tareas, botones (Completar, Eliminar, Completar todas, Reiniciar), etiqueta de pendientes e historial.
- Aceptado: la estructura completa. La ventana abrió bien y comprobé que el filtro "alta" muestra solo las tareas de prioridad alta, que Completar marca la tarea como hecha y que el historial registra cada evento.
- Modificado: nada en esta etapa.
- Rechazado: nada.
- Por qué: los botones solo llaman a funciones de logica.py. Identifiqué como impuras

_refrescar, _id_elegidoy _titulo_valido, porque modifican widgets, leen la pantalla y muestran mensajes.

Interacción 4: contador de pendientes según el filtro

- Prompt: "cómo se hace eso de refrescar en las tareas", "cómo cambio eso o está bien así?", "dónde cambio eso de interfaz" y "en qué parte cambio esto" (sobre contar_pendientes).
- Fragmento sugerido: cambiar contar_pendientesy contar_pendientes_imperativo para que reciban una tupla de tareas, y en _refrescar llamar a contar_pendientes(visibles).
- Aceptado: el cambio, porque hace que el número coincida con lo que se ve en la tabla.
- Error detectado: al ejecutar python logica.pyapareció AttributeError: 'tuple' object has no attribute 'tareas'. Había cambiado el parámetro pero dentro del reduce seguía estado.tareas.
- Modificado: reemplacé estado.tareas por tareas dentro de contar_pendientes, guardé y volví a ejecutar. Después probé main.pyy el contador funcionó.
- Rechazado: nada.
- Por qué: una función pura debe recibir solo lo que necesita. Contar tareas no requiere conocer todo el Estado.
- Lo que aprendí: un error de Python indica el archivo, la línea y el nombre exacto que falla. Al cambiar el parámetro de una función hay que revisar su cuerpo y también todos los lugares donde se la llama.
- Segundo error: después de arreglar logica.py, al ejecutar main.py apareció TypeError: reduce() arg 2 must support iteration, porque en interfaz.py seguía llamando contar_pendientes(self.estado). Lo corregí cambiándolo por contar_pendientes(visibles) y volví a ejecutar.